Android App Development
Hardware and Sensors

Part—5

I'm back with another article on Android application development. This time, we will discuss how to access some of the phone's hardware components like the camera flash, accelerometer, etc, with two simple applications to better understand the concepts and also create a real-world app.

I assume my readers already know the basics, so I will dwell more on new concepts and processes. Our first app is a simple *TorchLight* example; you can find many demos on the Internet that change the screen background to white, emulating a torch with the light from the screen. Here, we will access the camera flash (now found on most camera phones) to build our example. Figure 1 is a design diagram. Before moving to the implementation, be aware that this app will only work for phones with a flash—and camera—so it should not be listed in the market for phones lacking this feature.

As per the design, you first need to access the phone's flash. Flashlight is a component of *android.hardware.camera*. Once you get access to it, get the parameters from the existing *Flashlight*, then override them for your functionality, and then release the camera object for other applications to use.

Torch app implementation

Create an Android project named *SimpleFlashLight*. To use a hardware component, you need to claim it, in the manifest file. Since *Flashlight* comes under *Camera*, your app needs permission to access both:

```
<uses-permission android:name="android.permission.CAMERA"></
uses-permission>
<uses-permission android:name="android.permission.FLASHLIGHT"></
uses-permission>
```

To restrict app availability to only phones with the required hardware, add the following XML to allow filtering of devices that this app will be compatible with:

```
<uses-feature android:name="android.hardware.Camera"
android:required="true"></uses-feature>
<uses-feature android:name="android.hardware.camera.FLASHLIGHT"
      android:required="true"></uses-feature>
```

Now, let's design a simple layout with a *TextView* as the tile, and a simple *ToggleButton* to switch the light on/off:

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/
android"
      android:orientation="vertical" android:layout_width="fill_
parent"
```